



L8A: Mutable Data, Lists, and Loops

CS1101S: Programming Methodology

Martin Henz

7th October, 2026

Version 1.1 — Built: 2026-10-06 22:37:10 SGT



One Week To Go: Sumobot 2026

Thursday, 15 October
6–9pm at COM3 MPH

**Refreshments, prizes,
swag, and a guest talk
about Jane Street.**

Sponsored by



Jane Street®





Sign Up Now!

Applications are open!

Thursday, 15 October, 6–9pm
COM3 MPH

Competitors and spectators
welcome.



Scan the QR code or visit:

`forms.gle/bCeZSFc2h6ZRhjWJ8`



Outline

State

Mutable Data

Lists

Loops

Lists and Loops



Course overview

- ▶ Unit 1—Functions (textbook Chapter 1)
 - Getting acquainted with the elements of programming, using **functional abstraction**
 - Learning to read programs, and using the **substitution model**
 - Example applications: Runes, curves
- ▶ Unit 2—Data (textbook Chapter 2)
 - Getting familiar with data: pairs, linked lists, trees, streams
 - Searching in linked lists and trees, sorting of linked lists
 - Example application: sound processing



Course overview, continued

- ▶ **Unit 3—State (parts of textbook Chapter 3)**
 - Programming with stateful abstractions
 - Lists, loops, searching in and sorting of lists
 - Reading programs using the environment model / CSE machine
 - Example applications: robotics, video processing
- ▶ **Unit 4—Beyond (parts of textbook Chapters 3 and 4)**
 - Memoization, Memoized Streams
 - Metalinguistic abstraction



Python §3



Python §3

- ▶ Python §3 adds reassignment, mutable pairs, and lists.



Python §3

- ▶ Python §3 adds reassignment, mutable pairs, and lists.
- ▶ Our pairs (the components of linked lists) are actually Python lists in Python §3.



Python §3

- ▶ Python §3 adds reassignment, mutable pairs, and lists.
- ▶ Our pairs (the components of linked lists) are actually Python lists in Python §3.
- ▶ Python §3 adds `while`, `for`, `break`, and `continue`.



Python §3

- ▶ Python §3 adds reassignment, mutable pairs, and lists.
- ▶ Our pairs (the components of linked lists) are actually Python lists in Python §3.
- ▶ Python §3 adds `while`, `for`, `break`, and `continue`.
- ▶ [Python §3 specification](#) and [Source Academy Playground](#)



Outline

State

Mutable Data

Lists

Loops

Lists and Loops

λ

So far ...



So far ...

- ▶ ...our functions are mostly **pure**.
 - The same inputs give the same result.
 - Calling a function does not change existing data.



So far ...

- ▶ ...our functions are mostly **pure**.
 - The same inputs give the same result.
 - Calling a function does not change existing data.
- ▶ The substitution model helps us reason about such computations.



So far ...

- ▶ ...our functions are mostly **pure**.
 - The same inputs give the same result.
 - Calling a function does not change existing data.
- ▶ The substitution model helps us reason about such computations.
- ▶ Today: functions and data change while the program runs!



Review: declaration assignment

```
name = expression
```



Review: declaration assignment

```
name = expression
```

- ▶ The right-hand `expression` is evaluated first.



Review: declaration assignment

```
name = expression
```

- ▶ The right-hand `expression` is evaluated first.
- ▶ Its value is bound to `name` in the current scope.



Review: declaration assignment

```
name = expression
```

- ▶ The right-hand `expression` is evaluated first.
- ▶ Its value is bound to `name` in the current scope.
- ▶ In Python §1 and §2, a binding cannot be reassigned.



Review: declaration assignment

```
name = expression
```

- ▶ The right-hand `expression` is evaluated first.
- ▶ Its value is bound to `name` in the current scope.
- ▶ In Python §1 and §2, a binding cannot be reassigned.
- ▶ Python §3 allows an existing variable or parameter to be reassigned.



Reassignment

```
balance = 100           # declaration assignment  
balance = balance - 20  # reassignment  
print(balance)
```





Reassignment

```
balance = 100           # declaration assignment  
balance = balance - 20   # reassignment  
print(balance)
```

- Evaluate `balance - 20` using the old value: $100 - 20 = 80$.



Reassignment

```
balance = 100           # declaration assignment  
balance = balance - 20   # reassignment  
print(balance)
```

- ▶ Evaluate `balance - 20` using the old value: $100 - 20 = 80$.
- ▶ Update the binding of `balance` to the new value.



A simple bank account

```
balance = 100
def withdraw(amount):
    global balance
    if balance >= amount:
        balance = balance - amount
        return balance
    else:
        return 'Insufficient funds'

print(withdraw(25))
```





A simple bank account

```
balance = 100
def withdraw(amount):
    global balance
    if balance >= amount:
        balance = balance - amount
        return balance
    else:
        return 'Insufficient funds'

print(withdraw(25))
```

- ▶ withdraw operates on a global balance.

A simple bank account

```
balance = 100
def withdraw(amount):
    global balance
    if balance >= amount:
        balance = balance - amount
        return balance
    else:
        return 'Insufficient funds'

print(withdraw(25))
```

- ▶ `withdraw` operates on a global `balance`.
- ▶ `global balance` selects the global binding for reassignment.



A simple bank account

```
balance = 100
def withdraw(amount):
    global balance
    if balance >= amount:
        balance = balance - amount
        return balance
    else:
        return 'Insufficient funds'

print(withdraw(25))
```

- ▶ `withdraw` operates on a global `balance`.
- ▶ `global balance` selects the global binding for reassignment.
- ▶ Without `global`, `balance = ...` would be a declaration assignment, not a reassignment.



Functional and imperative programming



Functional and imperative programming

- ▶ A pure `factorial(5)` always returns 120.



Functional and imperative programming

- ▶ A pure `factorial(5)` always returns 120.
- ▶ Compare an account that starts with \$100:



Functional and imperative programming

- ▶ A pure `factorial(5)` always returns 120.
- ▶ Compare an account that starts with \$100:

Call	Returned balance	Printed message
<code>withdraw(40)</code>	60	
<code>withdraw(40)</code>	20	
<code>withdraw(40)</code>	20	Insufficient funds
<code>withdraw(15)</code>	5	



Functional and imperative programming

- ▶ A pure `factorial(5)` always returns 120.
- ▶ Compare an account that starts with \$100:

Call	Returned balance	Printed message
<code>withdraw(40)</code>	60	
<code>withdraw(40)</code>	20	
<code>withdraw(40)</code>	20	Insufficient funds
<code>withdraw(15)</code>	5	

The result depends on earlier withdrawals, not just the argument.

λ

State



State

- ▶ Identical calls to `withdraw` can produce different results.



State

- ▶ Identical calls to `withdraw` can produce different results.
- ▶ The account remembers its current balance: this is its **state**.



State

- ▶ Identical calls to `withdraw` can produce different results.
- ▶ The account remembers its current balance: this is its **state**.
- ▶ Reassignment lets us update state held by a function.



Protecting the balance

```
def make_account(balance):  
    def withdraw(amount):  
        nonlocal balance  
        if balance >= amount:  
            balance = balance - amount  
        else:  
            print('Insufficient funds')  
        return balance  
    return withdraw
```





Protecting the balance

```
def make_account(balance):  
    def withdraw(amount):  
        nonlocal balance  
        if balance >= amount:  
            balance = balance - amount  
        else:  
            print('Insufficient funds')  
        return balance  
    return withdraw
```

- ▶ `make_account` returns a function with its own stored balance.



Protecting the balance

```
def make_account(balance):  
    def withdraw(amount):  
        nonlocal balance  
        if balance >= amount:  
            balance = balance - amount  
        else:  
            print('Insufficient funds')  
        return balance  
    return withdraw
```

- ▶ `make_account` returns a function with its own stored balance.
- ▶ `nonlocal balance` selects the enclosing binding for reassignment.



Protecting the balance

```
def make_account(balance):  
    def withdraw(amount):  
        nonlocal balance  
        if balance >= amount:  
            balance = balance - amount  
        else:  
            print('Insufficient funds')  
        return balance  
    return withdraw
```

- ▶ `make_account` returns a function with its own stored balance.
- ▶ `nonlocal balance` selects the enclosing binding for reassignment.
- ▶ Without `nonlocal`, `balance = ...` would be a declaration assignment, not a reassignment.



Multiple accounts

```
W1 = make_account(100)
W2 = make_account(100)
print(W1(50))
print(W2(70))
print(W2(40))
print(W1(40))
```





Multiple accounts

```
W1 = make_account(100)
W2 = make_account(100)
print(W1(50))
print(W2(70))
print(W2(40))
print(W1(40))
```

- Each call to `make_account` creates a separate balance binding.



Multiple accounts

```
W1 = make_account(100)
W2 = make_account(100)
print(W1(50))
print(W2(70))
print(W2(40))
print(W1(40))
```

- ▶ Each call to `make_account` creates a separate balance binding.
- ▶ A withdrawal from `W1` does not change `W2`.



Reassignment: pros



Reassignment: pros

- ▶ Reassignment allows us to create objects with state



Reassignment: pros

- ▶ Reassignment allows us to create objects with state
- ▶ State allows objects to behave differently over time



Reassignment: cons



Reassignment: cons

- ▶ Calls may depend on earlier calls, so execution order matters.



Reassignment: cons

- ▶ Calls may depend on earlier calls, so execution order matters.
- ▶ Shared state can make debugging and checking correctness harder.



Reassignment: cons

- ▶ Calls may depend on earlier calls, so execution order matters.
- ▶ Shared state can make debugging and checking correctness harder.
- ▶ A fixed substitution of names by values no longer explains evaluation.



Why fixed substitution fails

```
balance = 100
print(balance)
balance = balance - 20
print(balance)
```





Why fixed substitution fails

```
balance = 100
print(balance)
balance = balance - 20
print(balance)
```



- ▶ The two occurrences of `balance` in `print` have different values.



Why fixed substitution fails

```
balance = 100  
print(balance)  
balance = balance - 20  
print(balance)
```



- ▶ The two occurrences of `balance` in `print` have different values.
- ▶ Replacing every occurrence by 100 gives the wrong result.



Why fixed substitution fails

```
balance = 100  
print(balance)  
balance = balance - 20  
print(balance)
```



- ▶ The two occurrences of `balance` in `print` have different values.
- ▶ Replacing every occurrence by 100 gives the wrong result.
- ▶ We must track the binding as it changes.



Why fixed substitution fails

```
balance = 100
print(balance)
balance = balance - 20
print(balance)
```



- ▶ The two occurrences of `balance` in `print` have different values.
- ▶ Replacing every occurrence by 100 gives the wrong result.
- ▶ We must track the binding as it changes.
- ▶ Our substitution model fails. **We need a new model for understanding imperative programs: L8B**



Outline

State

Mutable Data

Lists

Loops

Lists and Loops



Mutable data



Mutable data

- ▶ **Reassignment:** change the value bound to a name.



Mutable data

- ▶ **Reassignment:** change the value bound to a name.
- ▶ **Mutation:** change an existing data structure.



Mutable data

- ▶ **Reassignment:** change the value bound to a name.
- ▶ **Mutation:** change an existing data structure.
- ▶ In Python §2, we used pair constructors and selectors, but no mutators.



Mutable data

- ▶ **Reassignment:** change the value bound to a name.
- ▶ **Mutation:** change an existing data structure.
- ▶ In Python §2, we used pair constructors and selectors, but no mutators.
- ▶ Python §3 adds mutators for pairs and element reassignment for Python lists.



Mutable pairs



Mutable pairs

- ▶ `pair(x, y)` constructs a pair.



Mutable pairs

- ▶ `pair(x, y)` constructs a pair.
- ▶ `head(p)` and `tail(p)` select its two components.

Mutable pairs

- ▶ `pair(x, y)` constructs a pair.
- ▶ `head(p)` and `tail(p)` select its two components.
- ▶ `set_head(p, x)` replaces its head with `x`.

Mutable pairs

- ▶ `pair(x, y)` constructs a pair.
- ▶ `head(p)` and `tail(p)` select its two components.
- ▶ `set_head(p, x)` replaces its head with `x`.
- ▶ `set_tail(p, y)` replaces its tail with `y`.



Mutable pairs

- ▶ `pair(x, y)` constructs a pair.
- ▶ `head(p)` and `tail(p)` select its two components.
- ▶ `set_head(p, x)` replaces its head with `x`.
- ▶ `set_tail(p, y)` replaces its tail with `y`.
- ▶ The pair itself remains the same object.



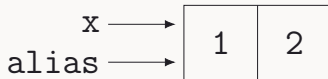
Mutable pairs

- ▶ `pair(x, y)` constructs a pair.
- ▶ `head(p)` and `tail(p)` select its two components.
- ▶ `set_head(p, x)` replaces its head with `x`.
- ▶ `set_tail(p, y)` replaces its tail with `y`.
- ▶ The pair itself remains the same object.
- ▶ Any other name referring to that pair observes the change.



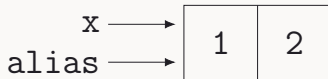
set_head example

```
x = pair(1, 2)
alias = x
set_head(x, 9)
print(head(x))
print(tail(x))
print(head(alias))
```



set_head example

```
x = pair(1, 2)
alias = x
set_head(x, 9)
print(head(x))
print(tail(x))
print(head(alias))
```

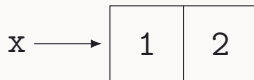


Both `x` and `alias` refer to the changed pair.



set_tail example

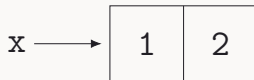
```
x = pair(1, 2)
set_tail(x, llist(3, 4))
print(head(x))
print(head(tail(x)))
```





set_tail example

```
x = pair(1, 2)
set_tail(x, llist(3, 4))
print(head(x))
print(head(tail(x)))
```



The head is unchanged; the tail now points to a linked list.



Recall the append function

```
def append(xs, ys):  
    return (ys if is_none(xs)  
           else pair(head(xs), append(tail(xs), ys)))
```



Recall the append function

```
def append(xs, ys):  
    return (ys if is_none(xs)  
            else pair(head(xs), append(tail(xs), ys)))
```

- One fresh pair for each pair in `xs`.

Recall the append function

```
def append(xs, ys):  
    return (ys if is_none(xs)  
           else pair(head(xs), append(tail(xs), ys)))
```

- ▶ One fresh pair for each pair in `xs`.
- ▶ At the base case, return `ys` itself, not a copy.

Recall the append function

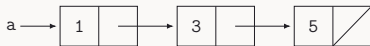
```
def append(xs, ys):  
    return (ys if is_none(xs)  
           else pair(head(xs), append(tail(xs), ys)))
```

- ▶ One fresh pair for each pair in `xs`.
- ▶ At the base case, return `ys` itself, not a copy.
- ▶ The result shares its final segment with `ys`.



Example: create two linked lists

```
a = llist(1, 3, 5)  
b = llist(2, 4)
```

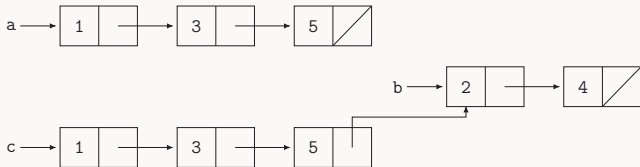


The two linked lists contain separate pairs.



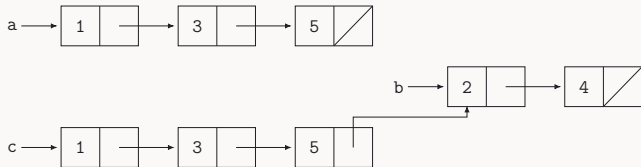
Example: append the linked lists

```
c = append(a, b)
```



Example: append the linked lists

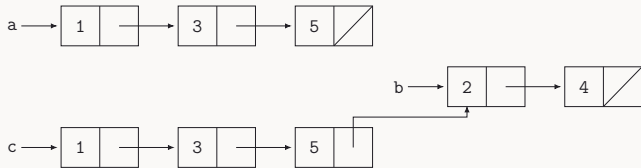
```
c = append(a, b)
```



- `append` creates new pairs for the elements of `a`.

Example: append the linked lists

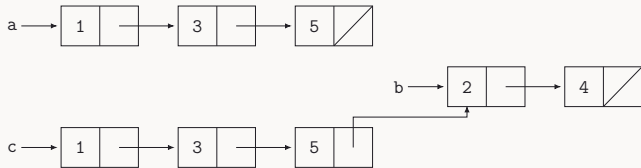
```
c = append(a, b)
```



- ▶ `append` creates new pairs for the elements of `a`.
- ▶ The final tail of `c` points into the original `b`.

Example: append the linked lists

```
c = append(a, b)
```

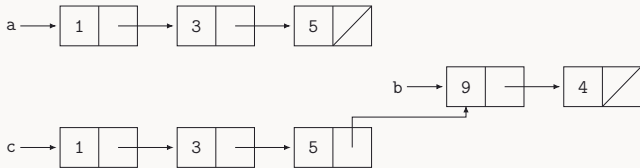


- ▶ `append` creates new pairs for the elements of **a**.
- ▶ The final tail of **c** points into the original **b**.
- ▶ The pairs of **b** are shared with **c**.



Example: mutate the shared head

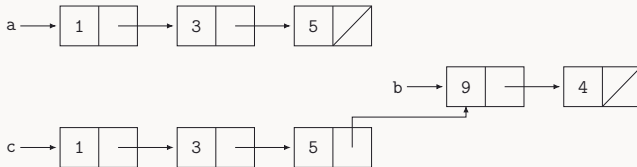
```
set_head(b, 9)
```





Example: mutate the shared head

```
set_head(b, 9)
```

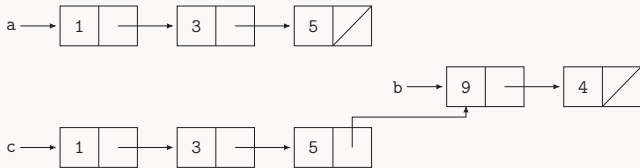


- Shared pair changes: both `b` and `c` now lead to 9.



Example: mutate the shared head

```
set_head(b, 9)
```

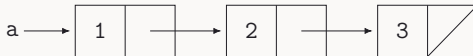


- ▶ Shared pair changes: both **b** and **c** now lead to 9.
- ▶ Mutating **a** will **not** affect **c**.



Another example

```
a = llist(1, 2, 3)
```

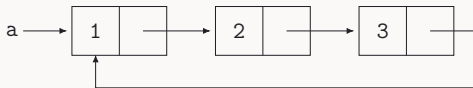


The last tail is `None`: following the tails eventually stops.



Another example: creating a cycle

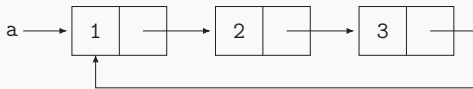
```
set_tail(tail(tail(a)), a)
```





Another example: creating a cycle

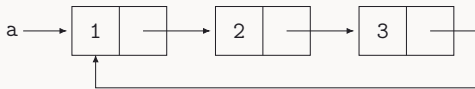
```
set_tail(tail(tail(a)), a)
```



- The last tail points back to the first pair: a **cycle**.

Another example: creating a cycle

```
set_tail(tail(tail(a)), a)
```



- ▶ The last tail points back to the first pair: a **cycle**.
- ▶ `length(a)` never reaches `None`; it does not terminate normally.



Destructive linked list processing



Destructive linked list processing

- ▶ Append two linked lists **without allocating new pairs**.



Destructive linked list processing

- ▶ Append two linked lists **without allocating new pairs**.
- ▶ Reuse the input pairs by changing the last tail of the first linked list.



Destructive linked list processing

- ▶ Append two linked lists **without allocating new pairs**.
- ▶ Reuse the input pairs by changing the last tail of the first linked list.
- ▶ Return the first linked list's head, or the second linked list if the first is empty.



Destructive linked list processing

- ▶ Append two linked lists **without allocating new pairs**.
- ▶ Reuse the input pairs by changing the last tail of the first linked list.
- ▶ Return the first linked list's head, or the second linked list if the first is empty.
- ▶ Unlike `append`, this changes the first input.



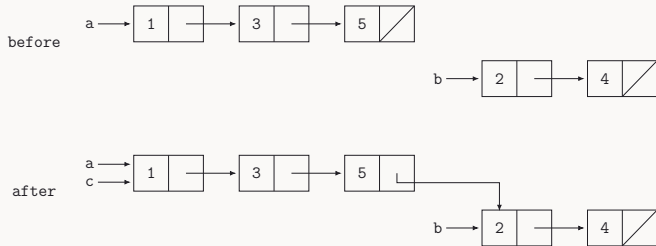
Destructive linked list processing

- ▶ Append two linked lists **without allocating new pairs**.
- ▶ Reuse the input pairs by changing the last tail of the first linked list.
- ▶ Return the first linked list's head, or the second linked list if the first is empty.
- ▶ Unlike `append`, this changes the first input.
- ▶ Assume the inputs are finite, acyclic, and share no pairs. Otherwise, linking them can create a cycle.



Destructive append: example

```
a = llist(1, 3, 5)
b = llist(2, 4)
c = d_append(a, b)
print(a)
print(b)
print(c)
```





Destructive append: implementation

```
def d_append(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        set_tail(xs, d_append(tail(xs), ys))  
        return xs
```



Destructive append: implementation

```
def d_append(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        set_tail(xs, d_append(tail(xs), ys))  
        return xs
```

- Recurse to the end; replace tail pointers on the way back.

Destructive append: implementation

```
def d_append(xs, ys):  
    if is_none(xs):  
        return ys  
    else:  
        set_tail(xs, d_append(tail(xs), ys))  
        return xs
```

- ▶ Recurse to the end; replace tail pointers on the way back.
- ▶ No calls to `pair`: no new pairs are constructed.



Destructive map: example

```
a = llist(1, 2, 3)
d_map(lambda x: x * 2, a)
a
```





Destructive map: example

```
a = llist(1, 2, 3)
d_map(lambda x: x * 2, a)
a
```



- Transform the heads of the existing pairs in place.

Destructive map: example

```
a = llist(1, 2, 3)
d_map(lambda x: x * 2, a)
a
```



- ▶ Transform the heads of the existing pairs in place.
- ▶ Any other reference to these pairs sees the changed values.



Destructive map: implementation

```
def d_map(fun, xs):  
    if not is_none(xs):  
        set_head(xs, fun(head(xs)))  
        d_map(fun, tail(xs))
```





Destructive map: implementation

```
def d_map(fun, xs):  
    if not is_none(xs):  
        set_head(xs, fun(head(xs)))  
        d_map(fun, tail(xs))
```



- Each head changes; the tail links remain unchanged.



Destructive map: implementation

```
def d_map(fun, xs):  
    if not is_none(xs):  
        set_head(xs, fun(head(xs)))  
        d_map(fun, tail(xs))
```

- ▶ Each head changes; the tail links remain unchanged.
- ▶ The function returns `None`; the useful result is its effect on `xs`.



Recap: state and mutable data



Recap: state and mutable data

- ▶ Reassignment updates a name's binding.



Recap: state and mutable data

- ▶ Resignment updates a name's binding.
- ▶ Mutators change existing pairs and linked lists.



Recap: state and mutable data

- ▶ Resignment updates a name's binding.
- ▶ Mutators change existing pairs and linked lists.
- ▶ Shared references observe the same mutations.



Recap: state and mutable data

- ▶ Resignment updates a name's binding.
- ▶ Mutators change existing pairs and linked lists.
- ▶ Shared references observe the same mutations.
- ▶ Ordinary `append` copies its first list's spine and shares its second.



Recap: state and mutable data

- ▶ Resignment updates a name's binding.
- ▶ Mutators change existing pairs and linked lists.
- ▶ Shared references observe the same mutations.
- ▶ Ordinary `append` copies its first list's spine and shares its second.
- ▶ Destructive operations reuse pairs, so check for sharing and cycles.



Outline

State

Mutable Data

Lists

Loops

Lists and Loops

λ

Pairs



Pairs

- ▶ Python §2 has **pairs**: collections of two items called head and tail.



Pairs

- ▶ Python §2 has **pairs**: collections of two items called head and tail.
- ▶ Box notation prints pairs: `pair(1, 2)` is shown as `[1, 2]`



Pairs

- ▶ Python §2 has **pairs**: collections of two items called head and tail.
- ▶ Box notation prints pairs: `pair(1, 2)` is shown as `[1, 2]`
- ▶ In Python §3 you can write box notation in your program: `[1, 2]` is a valid Python §3 program.

Pairs

- ▶ Python §2 has **pairs**: collections of two items called head and tail.
- ▶ Box notation prints pairs: `pair(1, 2)` is shown as `[1, 2]`
- ▶ In Python §3 you can write box notation in your program: `[1, 2]` is a valid Python §3 program.
- ▶ You can write `p[0]` for `head(p)`.

Pairs

- ▶ Python §2 has **pairs**: collections of two items called head and tail.
- ▶ Box notation prints pairs: `pair(1, 2)` is shown as `[1, 2]`
- ▶ In Python §3 you can write box notation in your program: `[1, 2]` is a valid Python §3 program.
- ▶ You can write `p[0]` for `head(p)`.
- ▶ You can write `p[1]` for `tail(p)`.

Pairs

- ▶ Python §2 has **pairs**: collections of two items called head and tail.
- ▶ Box notation prints pairs: `pair(1, 2)` is shown as `[1, 2]`
- ▶ In Python §3 you can write box notation in your program: `[1, 2]` is a valid Python §3 program.
- ▶ You can write `p[0]` for `head(p)`.
- ▶ You can write `p[1]` for `tail(p)`.

```
p = [1, 2]
print(p[0])
print(p[1])
```





Lists generalize pairs

Python **lists** are indexed sequences written as [...].



Lists generalize pairs

Python **lists** are indexed sequences written as [...].

```
seq = [10, 5, 8]
print(seq[0])
print(seq[2])
print(seq[-1])
```





Lists generalize pairs

Python **lists** are indexed sequences written as [...].

```
seq = [10, 5, 8]
print(seq[0])
print(seq[2])
print(seq[-1])
```



- Indices start at 0; negative indices count from the end.



Lists generalize pairs

Python **lists** are indexed sequences written as [...].

```
seq = [10, 5, 8]
print(seq[0])
print(seq[2])
print(seq[-1])
```



- ▶ Indices start at 0; negative indices count from the end.
- ▶ `seq[-1]` selects the last element.



Lists generalize pairs

Python **lists** are indexed sequences written as [...].

```
seq = [10, 5, 8]
print(seq[0])
print(seq[2])
print(seq[-1])
```



- ▶ Indices start at 0; negative indices count from the end.
- ▶ `seq[-1]` selects the last element.
- ▶ Pairs are two-element lists.



List element reassignment

```
seq = [10, 5, 8]
alias = seq
seq[0] = 20
print(seq)
print(alias)
```





List element reassignment

```
seq = [10, 5, 8]
alias = seq
seq[0] = 20
print(seq)
print(alias)
```

- ▶ Element reassignment changes the existing list.



List element reassignment

```
seq = [10, 5, 8]
alias = seq
seq[0] = 20
print(seq)
print(alias)
```



- ▶ Element reassignment changes the existing list.
- ▶ It does not create a new list or change its length.



List element reassignment

```
seq = [10, 5, 8]
alias = seq
seq[0] = 20
print(seq)
print(alias)
```



- ▶ Element reassignment changes the existing list.
- ▶ It does not create a new list or change its length.
- ▶ `alias` refers to the same list and sees the update.



List length

```
seq = [10, 5, 8]
my_list = []
print(len(seq))
print(len(my_list))
```





List length

```
seq = [10, 5, 8]
my_list = []
print(len(seq))
print(len(my_list))
```

- ▶ `len` returns the number of elements, in $\Theta(1)$ time.



List length

```
seq = [10, 5, 8]
my_list = []
print(len(seq))
print(len(my_list))
```



- ▶ `len` returns the number of elements, in $\Theta(1)$ time.
- ▶ For `seq`, the valid indices are -3, -2, -1, 0, 1, and 2.

List length

```
seq = [10, 5, 8]
my_list = []
print(len(seq))
print(len(my_list))
```

- ▶ `len` returns the number of elements, in $\Theta(1)$ time.
- ▶ For `seq`, the valid indices are -3, -2, -1, 0, 1, and 2.
- ▶ Reassigning `seq[10] = 99` raises an error `index out of range`.



Allocating a list

```
seq = [None] * 11  
seq[0] = 10  
seq[1] = 5  
seq[2] = 8  
seq[10] = 99  
len(seq)
```





Allocating a list

```
seq = [None] * 11
seq[0] = 10
seq[1] = 5
seq[2] = 8
seq[10] = 99
len(seq)
```

- ▶ `[None] * 11` creates a list with 11 existing slots.



Allocating a list

```
seq = [None] * 11
seq[0] = 10
seq[1] = 5
seq[2] = 8
seq[10] = 99
len(seq)
```

- ▶ `[None] * 11` creates a list with 11 existing slots.
- ▶ `*` repeats the elements of the list.



List example

```
things = [123, 'cat', 'orange']  
print(len(things))  
print(things[0])  
print(things[2])  
things[2] = 'apple'  
print(things[2])
```





List example

```
things = [123, 'cat', 'orange']  
print(len(things))  
print(things[0])  
print(things[2])  
things[2] = 'apple'  
print(things[2])
```



A list can contain values of different types.



Random access



Random access

- ▶ For a valid index, reading or reassigning a list element takes $\Theta(1)$ time.



Random access

- ▶ For a valid index, reading or reassigning a list element takes $\Theta(1)$ time.
- ▶ Reaching element i of a linked list takes $\Theta(i + 1)$ time.



Random access

- ▶ For a valid index, reading or reassigning a list element takes $\Theta(1)$ time.
- ▶ Reaching element i of a linked list takes $\Theta(i + 1)$ time.
- ▶ Creating `[None] * n` takes $\Theta(n)$ time and space.



Nested lists

```
table = [[1, 2, 3, 4],  
         [5, 6, 7, 8],  
         [9, 10, 11]]  
  
print(len(table))  
print(table[1][2])  
print(len(table[0]))  
print(len(table[2]))
```





Nested lists

```
table = [[1, 2, 3, 4],  
         [5, 6, 7, 8],  
         [9, 10, 11]]  
  
print(len(table))  
print(table[1][2])  
print(len(table[0]))  
print(len(table[2]))
```

- ▶ The outer list contains row lists.



Nested lists

```
table = [[1, 2, 3, 4],  
         [5, 6, 7, 8],  
         [9, 10, 11]]  
  
print(len(table))  
print(table[1][2])  
print(len(table[0]))  
print(len(table[2]))
```

- ▶ The outer list contains row lists.
- ▶ Rows need not have the same length.



Nested lists

```
table = [[1, 2, 3, 4],  
         [5, 6, 7, 8],  
         [9, 10, 11]]  
  
print(len(table))  
print(table[1][2])  
print(len(table[0]))  
print(len(table[2]))
```

- ▶ The outer list contains row lists.
- ▶ Rows need not have the same length.
- ▶ `table[r][c]` first selects row `r`, then its element `c`.



Building a list recursively

For a non-negative integer n , return a list containing 1 through n .

```
def list_1_to_n(n):  
    a = [None] * n  
  
    def iter(i):  
        if i < n:  
            a[i] = i + 1  
            iter(i + 1)  
  
    iter(0)  
    return a  
  
list_1_to_n(3)
```

The recursive helper fills one slot per call.



Mapping a list recursively

```
def map_list(f, arr):  
    def iter(i):  
        if i < len(arr):  
            arr[i] = f(arr[i])  
            iter(i + 1)  
  
    iter(0)  
  
seq = [3, 1, 5]  
map_list(lambda x: 2 * x, seq)  
seq
```

This is an in-place operation: `seq` becomes `[6, 2, 10]`.



Outline

State

Mutable Data

Lists

Loops

Lists and Loops



while loop

```
while condition:  
    statement
```



while loop

```
while condition:  
    statement
```

- ▶ Evaluate the Boolean condition **before** each iteration.



while loop

```
while condition:  
    statement
```

- ▶ Evaluate the Boolean condition **before** each iteration.
- ▶ If it is `True`, execute the body and test again.



while loop

```
while condition:  
    statement
```

- ▶ Evaluate the Boolean condition **before** each iteration.
- ▶ If it is `True`, execute the body and test again.
- ▶ If it is `False`, leave the loop.



while loop

```
while condition:  
    statement
```

- ▶ Evaluate the Boolean condition **before** each iteration.
- ▶ If it is **True**, execute the body and test again.
- ▶ If it is **False**, leave the loop.
- ▶ The body may run zero times.



while loop

```
while condition:  
    statement
```

- ▶ Evaluate the Boolean condition **before** each iteration.
- ▶ If it is **True**, execute the body and test again.
- ▶ If it is **False**, leave the loop.
- ▶ The body may run zero times.
- ▶ Ensure each iteration makes progress towards termination.



Factorial: recursive definition

```
def factorial_r(n):  
    return 1 if n <= 1 else n * factorial_r(n - 1)  
  
print(factorial_r(0))  
print(factorial_r(5))
```





Factorial: recursive definition

```
def factorial_r(n):  
    return 1 if n <= 1 else n * factorial_r(n - 1)  
  
print(factorial_r(0))  
print(factorial_r(5))
```

- Domain: non-negative integers; $0! = 1$.

Factorial: recursive definition

```
def factorial_r(n):  
    return 1 if n <= 1 else n * factorial_r(n - 1)  
  
print(factorial_r(0))  
print(factorial_r(5))
```

- ▶ Domain: non-negative integers; $0! = 1$.
- ▶ Recursive process: pending multiplications grow with n .

Factorial: recursive definition

```
def factorial_r(n):  
    return 1 if n <= 1 else n * factorial_r(n - 1)  
  
print(factorial_r(0))  
print(factorial_r(5))
```

- ▶ Domain: non-negative integers; $0! = 1$.
- ▶ Recursive process: pending multiplications grow with n .
- ▶ Next: an accumulator that updates its state in a loop.



Factorial: a while loop

```
def factorial_w(n):  
    acc = 1  
    k = 0  
    while k < n:  
        k = k + 1  
        acc = acc * k  
    return acc  
  
print(factorial_w(0))  
print(factorial_w(5))
```



Factorial: a while loop

```
def factorial_w(n):  
    acc = 1  
    k = 0  
    while k < n:  
        k = k + 1  
        acc = acc * k  
    return acc  
  
print(factorial_w(0))  
print(factorial_w(5))
```

- At every evaluation of $k < n$ we have: $acc = k!$.

Factorial: a while loop

```
def factorial_w(n):  
    acc = 1  
    k = 0  
    while k < n:  
        k = k + 1  
        acc = acc * k  
    return acc  
  
print(factorial_w(0))  
print(factorial_w(5))
```

- ▶ At every evaluation of $k < n$ we have: $acc = k!$.
- ▶ On exit, $k = n$, so $acc = n!$.



for loop

```
for name in range(start, end):  
    statement
```



for loop

```
for name in range(start, end):  
    statement
```

- ▶ `start` and `end` are integer expressions.



for loop

```
for name in range(start, end):  
    statement
```

- ▶ start and end are integer expressions.
- ▶ Visit start, start + 1, ..., end - 1.



for loop

```
for name in range(start, end):  
    statement
```

- ▶ `start` and `end` are integer expressions.
- ▶ Visit `start`, `start + 1`, ..., `end - 1`.
- ▶ The end is **exclusive**.



for loop

```
for name in range(start, end):  
    statement
```

- ▶ `start` and `end` are integer expressions.
- ▶ Visit `start`, `start + 1`, ..., `end - 1`.
- ▶ The end is **exclusive**.
- ▶ With the default positive step, `start >= end` gives zero iterations.



for loops: ranges

```
for i in range(3):  
    print(i)  
for j in range(2, 5):  
    print(j)
```





for loops: ranges

```
for i in range(3):  
    print(i)  
for j in range(2, 5):  
    print(j)
```



- ▶ range(end) starts at 0.

for loops: ranges

```
for i in range(3):  
    print(i)  
for j in range(2, 5):  
    print(j)
```

- ▶ `range(end)` starts at 0.
- ▶ `range(start, end)` starts at `start`.



for loops: step size

```
for k in range(5, 0, -2):  
    print(k)
```





for loops: step size

```
for k in range(5, 0, -2):  
    print(k)
```



- ▶ `range(start, end, step)` changes the increment.

for loops: step size

```
for k in range(5, 0, -2):  
    print(k)
```



- ▶ `range(start, end, step)` changes the increment.
- ▶ A positive step moves upwards; a negative step moves downwards.



for loops: step size

```
for k in range(5, 0, -2):  
    print(k)
```



- ▶ `range(start, end, step)` changes the increment.
- ▶ A positive step moves upwards; a negative step moves downwards.
- ▶ The end remains exclusive, and the step cannot be zero.

for loops: step size

```
for k in range(5, 0, -2):  
    print(k)
```



- ▶ `range(start, end, step)` changes the increment.
- ▶ A positive step moves upwards; a negative step moves downwards.
- ▶ The end remains exclusive, and the step cannot be zero.
- ▶ The loop reassigns the next range value to its control variable each iteration.



Restrictions on loops in Python §3



Restrictions on loops in Python §3

- ▶ Python §3 restricts `for` loops to `range(...)`.



Restrictions on loops in Python §3

- ▶ Python §3 restricts `for` loops to `range(...)`.
- ▶ To visit list elements, iterate over their indices.



Restrictions on loops in Python §3

- ▶ Python §3 restricts `for` loops to `range(...)`.
- ▶ To visit list elements, iterate over their indices.
- ▶ To follow tail pointers in a linked list, use `while`.



Restrictions on loops in Python §3

- ▶ Python §3 restricts `for` loops to `range(...)`.
- ▶ To visit list elements, iterate over their indices.
- ▶ To follow tail pointers in a linked list, use `while`.
- ▶ The loop control variable is introduced by the `for` loop.



Restrictions on loops in Python §3

- ▶ Python §3 restricts `for` loops to `range(...)`.
- ▶ To visit list elements, iterate over their indices.
- ▶ To follow tail pointers in a linked list, use `while`.
- ▶ The loop control variable is introduced by the `for` loop.
- ▶ Use a fresh name for it in this sublanguage.



Factorial using for loop

```
def factorial_f(n):  
    acc = 1  
    for k in range(n):  
        acc = acc * (k + 1)  
    return acc  
  
print(factorial_f(0))  
print(factorial_f(5))
```





Factorial using for loop

```
def factorial_f(n):  
    acc = 1  
    for k in range(n):  
        acc = acc * (k + 1)  
    return acc  
  
print(factorial_f(0))  
print(factorial_f(5))
```

- The products are $1, 1 \cdot 2, \dots, 1 \cdot 2 \cdots n$.



Factorial using for loop

```
def factorial_f(n):  
    acc = 1  
    for k in range(n):  
        acc = acc * (k + 1)  
    return acc  
  
print(factorial_f(0))  
print(factorial_f(5))
```

- ▶ The products are $1, 1 \cdot 2, \dots, 1 \cdot 2 \cdots n$.
- ▶ For $n = 0$, there are no iterations; the result is 1.

Factorial using for loop

```
def factorial_f(n):  
    acc = 1  
    for k in range(n):  
        acc = acc * (k + 1)  
    return acc  
  
print(factorial_f(0))  
print(factorial_f(5))
```

- ▶ The products are $1, 1 \cdot 2, \dots, 1 \cdot 2 \cdots n$.
- ▶ For $n = 0$, there are no iterations; the result is 1.
- ▶ Both loop versions take $\Theta(n)$ time and $\Theta(1)$ auxiliary space.



Linked list length: recursion

```
def length(xs):  
    return (0 if is_none(xs)  
           else 1 + length(tail(xs)))  
  
length(l1list(1, 2, 3))
```





Linked list length: recursion

```
def length(xs):  
    return (0 if is_none(xs)  
           else 1 + length(tail(xs)))  
  
length(l1list(1, 2, 3))
```

- This counts pairs in a finite, acyclic linked list.



Linked list length: recursion

```
def length(xs):  
    return (0 if is_none(xs)  
            else 1 + length(tail(xs)))  
  
length(l1list(1, 2, 3))
```

- ▶ This counts pairs in a finite, acyclic linked list.
- ▶ Each recursive call leaves a pending addition.



Linked list length: recursion

```
def length(xs):  
    return (0 if is_none(xs)  
           else 1 + length(tail(xs)))  
  
length(l1list(1, 2, 3))
```

- ▶ This counts pairs in a finite, acyclic linked list.
- ▶ Each recursive call leaves a pending addition.
- ▶ $\Theta(n)$ time and $\Theta(n)$ auxiliary space.



Linked list length: a while loop

```
def length_loop(xs):  
    count = 0  
    p = xs  
    while not is_none(p):  
        count = count + 1  
        p = tail(p)  
    return count  
  
print(length_loop(None))  
print(length_loop(llist(1, 2, 3)))
```



Linked list length: a while loop

```
def length_loop(xs):  
    count = 0  
    p = xs  
    while not is_none(p):  
        count = count + 1  
        p = tail(p)  
    return count  
  
print(length_loop(None))  
print(length_loop(l1ist(1, 2, 3)))
```

- ▶ Follow tail pointers until `None`; the original list is unchanged.



Linked list length: a while loop

```
def length_loop(xs):  
    count = 0  
    p = xs  
    while not is_none(p):  
        count = count + 1  
        p = tail(p)  
    return count  
  
print(length_loop(None))  
print(length_loop(llist(1, 2, 3)))
```

- ▶ Follow tail pointers until `None`; the original list is unchanged.
- ▶ $\Theta(n)$ time and $\Theta(1)$ auxiliary space.



Nested loops

A loop can contain another loop. The inner loop runs completely for each iteration of the outer loop.

```
for i in range(1, 3):  
    for j in range(1, 4):  
        print(i * 10 + j)
```



Output

11
12
13
21
22
23



Nested loops

A loop can contain another loop. The inner loop runs completely for each iteration of the outer loop.

```
for i in range(1, 3):  
    for j in range(1, 4):  
        print(i * 10 + j)
```



Output

11
12
13
21
22
23

Each output `ij` shows the current values of `i` and `j`. For `i = 1`, `j` runs through 1, 2, 3 before `i` becomes 2.



The break statement

break exits the innermost enclosing loop.

```
for i in range(1, 5):  
    print(str(i) + ' here')  
    if i == 2:  
        break  
    print(str(i) + ' there')  
print('OK')
```

Output

```
1 here  
1 there  
2 here  
OK
```



The break statement

break exits the innermost enclosing loop.

```
for i in range(1, 5):  
    print(str(i) + ' here')  
    if i == 2:  
        break  
    print(str(i) + ' there')  
print('OK')
```

Output

```
1 here  
1 there  
2 here  
OK
```

At `i = 2`, the second print is skipped and the loop ends.



The continue statement

continue skips the rest of this iteration, but not the loop.

```
for i in range(1, 5):  
    print(str(i) + ' here')  
    if i == 2:  
        continue  
    print(str(i) + ' there')  
print('OK')
```

Output

```
1 here  
1 there  
2 here  
3 here  
3 there  
4 here  
4 there  
OK
```



break and continue: which loop?



break and continue: which loop?

- ▶ break: leave the loop immediately.



break and continue: which loop?

- ▶ `break`: leave the loop immediately.
- ▶ `continue`: proceed to the next iteration.



break and continue: which loop?

- ▶ `break`: leave the loop immediately.
- ▶ `continue`: proceed to the next iteration.
- ▶ In a `for` loop, the next range value is reassigned automatically.



break and continue: which loop?

- ▶ `break`: leave the loop immediately.
- ▶ `continue`: proceed to the next iteration.
- ▶ In a `for` loop, the next range value is reassigned automatically.
- ▶ In a `while` loop, ensure `continue` does not skip the update needed for progress.



break and continue: which loop?

- ▶ `break`: leave the loop immediately.
- ▶ `continue`: proceed to the next iteration.
- ▶ In a `for` loop, the next range value is reassigned automatically.
- ▶ In a `while` loop, ensure `continue` does not skip the update needed for progress.
- ▶ In nested loops, both statements affect only the innermost enclosing loop.



Outline

State

Mutable Data

Lists

Loops

Lists and Loops



Mapping a list with a for loop

```
def map_list(f, arr):  
    for i in range(len(arr)):  
        arr[i] = f(arr[i])  
  
seq = [3, 1, 5]  
map_list(lambda x: 2 * x, seq)  
seq
```





Mapping a list with a for loop

```
def map_list(f, arr):  
    for i in range(len(arr)):  
        arr[i] = f(arr[i])  
  
seq = [3, 1, 5]  
map_list(lambda x: 2 * x, seq)  
seq
```

- ▶ `range(len(arr))` visits every valid non-negative index.



Mapping a list with a for loop

```
def map_list(f, arr):  
    for i in range(len(arr)):  
        arr[i] = f(arr[i])  
  
seq = [3, 1, 5]  
map_list(lambda x: 2 * x, seq)  
seq
```

- ▶ `range(len(arr))` visits every valid non-negative index.
- ▶ As before, the list is changed in place.



Building a zero matrix

For non-negative dimensions, build a `rows` by `cols` zero matrix.

```
def zero_matrix(rows, cols):  
    M = [None] * rows  
    for r in range(rows):  
        M[r] = [None] * cols  
        for c in range(cols):  
            M[r][c] = 0  
    return M
```

```
zero_matrix(2, 3)
```



Building a zero matrix

For non-negative dimensions, build a `rows` by `cols` zero matrix.

```
def zero_matrix(rows, cols):  
    M = [None] * rows  
    for r in range(rows):  
        M[r] = [None] * cols  
        for c in range(cols):  
            M[r][c] = 0  
    return M
```

```
zero_matrix(2, 3)
```

- ▶ A fresh row list is created on each outer iteration.



Building a zero matrix

For non-negative dimensions, build a `rows` by `cols` zero matrix.

```
def zero_matrix(rows, cols):  
    M = [None] * rows  
    for r in range(rows):  
        M[r] = [None] * cols  
        for c in range(cols):  
            M[r][c] = 0  
    return M
```

```
zero_matrix(2, 3)
```

- ▶ A fresh row list is created on each outer iteration.
- ▶ Do not use `[[0] * cols] * rows`: that would share one row.



Zero matrix: sharing and cost

```
M = zero_matrix(2, 3)
M[0][0] = 9
M
```





Zero matrix: sharing and cost

```
M = zero_matrix(2, 3)
M[0][0] = 9
M
```



- Updating one row leaves the other row unchanged.

Zero matrix: sharing and cost

```
M = zero_matrix(2, 3)
M[0][0] = 9
M
```



- ▶ Updating one row leaves the other row unchanged.
- ▶ Allocate the outer list: $\Theta(\text{rows})$.

Zero matrix: sharing and cost

```
M = zero_matrix(2, 3)
M[0][0] = 9
M
```

- ▶ Updating one row leaves the other row unchanged.
- ▶ Allocate the outer list: $\Theta(\text{rows})$.
- ▶ Allocate and fill each row: $\Theta(\text{cols})$ per row.

Zero matrix: sharing and cost

```
M = zero_matrix(2, 3)
M[0][0] = 9
M
```



- ▶ Updating one row leaves the other row unchanged.
- ▶ Allocate the outer list: $\Theta(\text{rows})$.
- ▶ Allocate and fill each row: $\Theta(\text{cols})$ per row.
- ▶ Total time and result space: $\Theta(\text{rows} + \text{rows} \cdot \text{cols})$.

Zero matrix: sharing and cost

```
M = zero_matrix(2, 3)
M[0][0] = 9
M
```



- ▶ Updating one row leaves the other row unchanged.
- ▶ Allocate the outer list: $\Theta(\text{rows})$.
- ▶ Allocate and fill each row: $\Theta(\text{cols})$ per row.
- ▶ Total time and result space: $\Theta(\text{rows} + \text{rows} \cdot \text{cols})$.
- ▶ Same as: $\Theta(\text{rows} \cdot \text{cols})$.



Recap: lists and loops



Recap: lists and loops

- ▶ Lists provide constant-time access to existing elements.



Recap: lists and loops

- ▶ Lists provide constant-time access to existing elements.
- ▶ `while` loops express repeated tests and updates.



Recap: lists and loops

- ▶ Lists provide constant-time access to existing elements.
- ▶ `while` loops express repeated tests and updates.
- ▶ `for` loops express iteration over integer ranges.



Recap: lists and loops

- ▶ Lists provide constant-time access to existing elements.
- ▶ `while` loops express repeated tests and updates.
- ▶ `for` loops express iteration over integer ranges.
- ▶ `break` exits a loop; `continue` skips the rest of an iteration.



Recap: lists and loops

- ▶ Lists provide constant-time access to existing elements.
- ▶ `while` loops express repeated tests and updates.
- ▶ `for` loops express iteration over integer ranges.
- ▶ `break` exits a loop; `continue` skips the rest of an iteration.
- ▶ Nested loops can process nested lists.

λ

Summary



Summary

- ▶ Reassignment changes bindings; mutation changes existing data.



Summary

- ▶ Reassignment changes bindings; mutation changes existing data.
- ▶ Stateful functions retain information between calls.



Summary

- ▶ Reassignment changes bindings; mutation changes existing data.
- ▶ Stateful functions retain information between calls.
- ▶ Shared references observe the same mutations.



Summary

- ▶ Reassignment changes bindings; mutation changes existing data.
- ▶ Stateful functions retain information between calls.
- ▶ Shared references observe the same mutations.
- ▶ The environment model tracks changing bindings.



Summary

- ▶ Reassignment changes bindings; mutation changes existing data.
- ▶ Stateful functions retain information between calls.
- ▶ Shared references observe the same mutations.
- ▶ The environment model tracks changing bindings.
- ▶ Lists provide random access; loops express iterative computations.



Summary

- ▶ Reassignment changes bindings; mutation changes existing data.
- ▶ Stateful functions retain information between calls.
- ▶ Shared references observe the same mutations.
- ▶ The environment model tracks changing bindings.
- ▶ Lists provide random access; loops express iterative computations.
- ▶ Next: use environments to explain evaluation with state.